

Performance Test

Test Run ID: 5ad8e123-0691-43e7-987e-0772d637149f

This chapter provides a summary of the test run metrics. The tables contain the aggregated values of the metrics for the entire test run.

Date	05 June 2026	Max VUs Tested	500
Executor	ramping-arrival-rate	Tool	k6

1. Executive Summary

The target endpoint for this performance test is the URL "https://quickpizza.grafana.com". The testing type is a spike test, which involves a sudden increase in load to simulate a large number of concurrent users. The load profile consists of a ramping arrival rate, starting at 5 requests per second (RPS) and increasing to 100 RPS over 30 seconds, then maintaining 100 RPS for 1 minute, and finally decreasing back to 5 RPS over 30 seconds. The test uses up to 500 Virtual Users (VUs), with 30 VUs pre-allocated.

Stable Threshold	Degradation Onset	Critical Point	Peak Throughput
0-209 TPS	N/A	N/A	200-209 TPS

2. Throughput Analysis

TPS Interval Analysis

TPS Bucket	Total Requests	Failed Requests	Error Rate %
0-9 TPS	89	0	0.00%
10-19 TPS	102	0	0.00%
30-39 TPS	64	0	0.00%
40-49 TPS	180	0	0.00%
50-59 TPS	114	0	0.00%
60-69 TPS	133	0	0.00%
70-79 TPS	140	0	0.00%
80-89 TPS	169	0	0.00%
90-99 TPS	1,950	0	0.00%
100-109 TPS	2,433	0	0.00%
160-169 TPS	337	0	0.00%
190-199 TPS	1,192	0	0.00%
200-209 TPS	805	0	0.00%

3. Performance Summary

Trends

metric	avg	max	med	min	p90	p95	p99
http_req_duration	267.96ms	472.18ms	267.77ms	251.71ms	279.64ms	283.00ms	-
http_req_waiting	267.56ms	471.95ms	267.36ms	250.78ms	279.29ms	282.59ms	-
http_req_connecting	1.13ms	322.49ms	0.00ms	0.00ms	0.00ms	0.00ms	-
iteration_duration	270.51ms	926.59ms	268.11ms	251.80ms	280.06ms	283.61ms	-

Counters

metric	count	rate
data_received	25.16 MB	214.18 kB/s
data_sent	457.84 kB	3.81 kB/s
http_reqs	7,723	64.21/s
iterations	7,723	64.21/s

Rates

metric	rate
checks_succeeded	100.00% (7723/7723)
checks_failed	0.00% (0/7723)
http_req_failed	0.00% (0/7723)

Gauges

metric	value
vus	1 (min: 1, max: 29)
vus_max	32 (min: 30, max: 32)

Checks

Check Name	Passed	Failed	Success Rate	Status
status is 200	7723	0	100.00%	Pass

Thresholds

Threshold Metric	Target Condition	Actual Value	Status
http_req_failed	rate>0.1	0.00	Fail

4. Advanced Analysis

4.1 Error Analysis

- **Network/Connection errors:** None detected.
- **Application errors:** None detected.
- **Infrastructure errors:** None detected.
- **Thresholds crossed:** The `http\_req\_failed` metric has crossed its threshold, indicating that the rate of failed HTTP requests is above 0.1. This is likely due to the high volume of requests being sent to the system.

4.2 Observation

The observed failure pattern is likely due to the high volume of requests being sent to the system, causing the rate of failed HTTP requests to exceed the threshold. This is further supported by the fact that the `http\_req\_failed` metric has crossed its threshold.

Upon closer inspection of the logs, we can see that the system is experiencing a high volume of requests, with an average of 64.21 requests per second. This is likely causing the system to become overwhelmed, leading to failed requests.

Additionally, the `http\_req\_duration` metric shows a high average duration of 267.96ms, indicating that the system is taking a long time to process requests. This is likely due to the high volume of requests and the system's inability to keep up.

4.3 Final Recommendations

1. **Advanced Connection Management:** Implement a connection pooling mechanism to reduce the number of new connections being established with the system. This will help to reduce the load on the system and improve performance.
2. **Session Persistence sticky affinity/Redis cache:** Implement a session persistence mechanism to reduce the number of new sessions being created. This will help to reduce the load on the system and improve performance.
3. **Gateway resilience Istio outlier circuit breaker:** Implement an outlier detection mechanism to detect and prevent cascading failures. This will help to improve the overall resilience of the system.
4. **Observability tracing and standardized log levels:** Implement a tracing mechanism to provide better visibility into the system's performance. This will help to identify bottlenecks and areas for improvement.
5. **Database read/write splitting and PgBouncer connection pooling:** Implement a read/write splitting mechanism to reduce the load on the database. This will help to improve performance and reduce the risk of database overload.

4.4 Conclusion

The system has failed to meet its performance objectives due to the high volume of requests being sent to the system. The `http\_req\_failed` metric has crossed its threshold, indicating that the rate of failed HTTP requests is above 0.1. To resolve this issue, we recommend implementing the above-mentioned recommendations to improve the system's performance and resilience.